

智驭能效（SmartEnergyMaster）— 技术选型 说明文档

版本：V1.0.0

日期：2026年6月8日

团队：第25组—智驭能效

所属机构：西北工业大学 软件学院

1 前言

1.1 编写目的

本文档旨在明确智驭能效（SmartEnergyMaster）项目全生命周期的技术栈、工具链及 AI 辅助开发方案，通过统一技术选型标准，保障项目开发的一致性、高效性与稳定性，为系统研发、测试、部署及运维提供技术依据，支撑项目达成"数字孪生→趋势预测→智能告警→优化调度"全链路闭环的核心目标。

1.2 适用范围

适用于本项目全生命周期技术实施场景（含需求分析、架构设计、编码开发、测试验证、部署运维阶段），覆盖研发、测试、运维、技术管理等相关团队，作为技术落地、工具配置、跨团队协同的执行依据。

1.3 技术选型原则

- 一致性**：统一技术栈版本，保障开发、测试、生产环境的兼容性；
- 适配性**：聚焦 PC 端大屏监控、移动端轻量查看等工业场景；
- 高效性**：选用成熟工具链 + AI 辅助编程（LLM诊断、Agent协作），降低开发成本；
- 稳定性**：优先选用企业级成熟技术组件（Spring Boot、TimescaleDB、Redis），保障生产环境运行可靠；
- 可扩展性**：保留功能扩展、系统集成的技术接口，支持后续业务迭代。

2 核心技术栈选型

2.1 前端技术栈

技术	版本	用途	选型理由
Vue 3	3.4+	前端框架	Composition API + 响应式系统，生态成熟
Vite	5.x	构建工具	极速冷启动、HMR热更新，替代 Webpack
Element Plus	2.x	UI 组件库	Vue 3 生态最成熟的企业级组件库
ECharts	5.x	数据可视化	支持仪表盘、折线图、热力图等工业监控图表
Pinia	2.x	状态管理	Vue 3 官方推荐，TypeScript 友好
Vue Router	4.x	路由管理	支持动态路由、路由守卫、懒加载
Axios	1.x	HTTP 客户端	拦截器机制、请求取消、超时控制

2.2 后端技术栈

技术	版本	用途	选型理由
Spring Boot	3.4.5	后端框架	企业级Java框架，生态完善，自动配置
Spring Security	6.x	安全认证	与Spring Boot深度集成，支持JWT无状态认证

技术	版本	用途	选型理由
MyBatis-Plus	3.5.5	ORM框架	简化CRUD，支持TimescaleDB PostgreSQL方言
JWT (jjwt + Hutool)	0.12.5 / 5.8.25	Token认证	无状态、可扩展、适合分布式部署
SpringDoc OpenAPI	2.8.6	API文档	Swagger UI交互式文档，Spring Boot 3.x兼容
Spring Data Redis	3.x	Redis集成	缓存、分布式锁、限流、Token黑名单
Jakarta Validation	3.x	参数校验	声明式校验，与Spring Boot自动集成

2.3 数据存储技术

技术	版本	用途	选型理由
TimescaleDB	pg14	时序数据库	PostgreSQL超表，自动分区，高效时序查询
Redis	7.x	缓存/队列/锁	高性能内存数据库，丰富数据结构
PostgreSQL JDBC	42.x	数据库驱动	TimescaleDB 基于PostgreSQL，完全兼容

2.4 AI 与模型技术

技术	版本	用途	选型理由
Python	3.10+	AI模型开发	AI/ML生态最成熟的语言
FastAPI	0.100+	AI服务框架	高性能异步框架，自动生成API文档
PyTorch	2.x	深度学习框架	LSTM/Transformer模型训练与推理
scikit-learn	1.x	数据处理	特征工程、数据标准化、评估指标
Pandas	2.x	数据处理	时序数据处理、CSV/Parquet导出
OpenAI SDK	1.x	LLM调用	兼容 DeepSeek/OpenAI 等主流LLM API
LangChain	0.3+	Agent框架	多Agent编排、Prompt模板、上下文管理

2.5 部署与运维技术

技术	版本	用途	选型理由
Docker	24+	容器化	环境一致性、快速部署
Docker Compose	v2	多服务编排	一键启动 TimescaleDB + Redis + 后端 + 前端 + AI服务
Nginx	1.25+	反向代理	前端静态资源托管 + API代理

技术	版本	用途	选型理由
OpenJDK	17	Java运行时	Spring Boot 3.x 要求 Java 17+

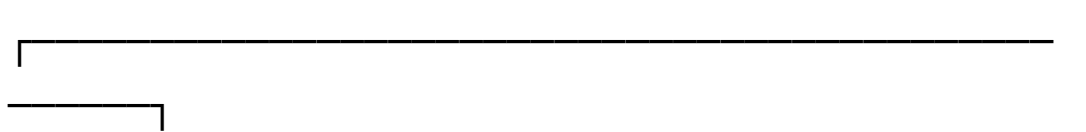
2.6 开发工具链

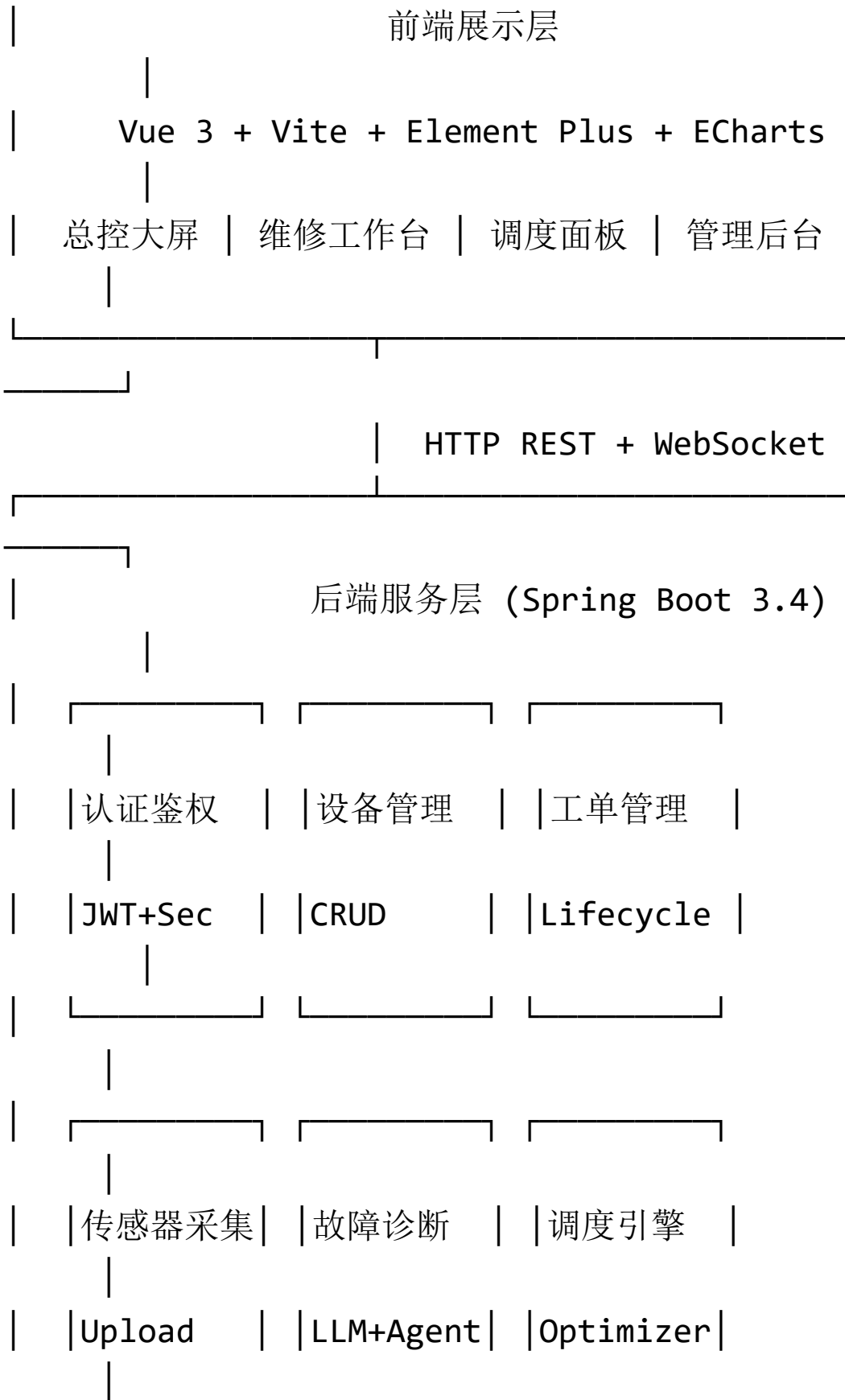
工具	用途
IntelliJ IDEA	后端 Java 开发
VS Code	前端 Vue 开发 + Python AI 开发
Git + GitHub	版本控制与代码托管
Maven	后端构建与依赖管理
npm	前端依赖管理
Postman	API 调试
Docker Desktop	本地容器环境

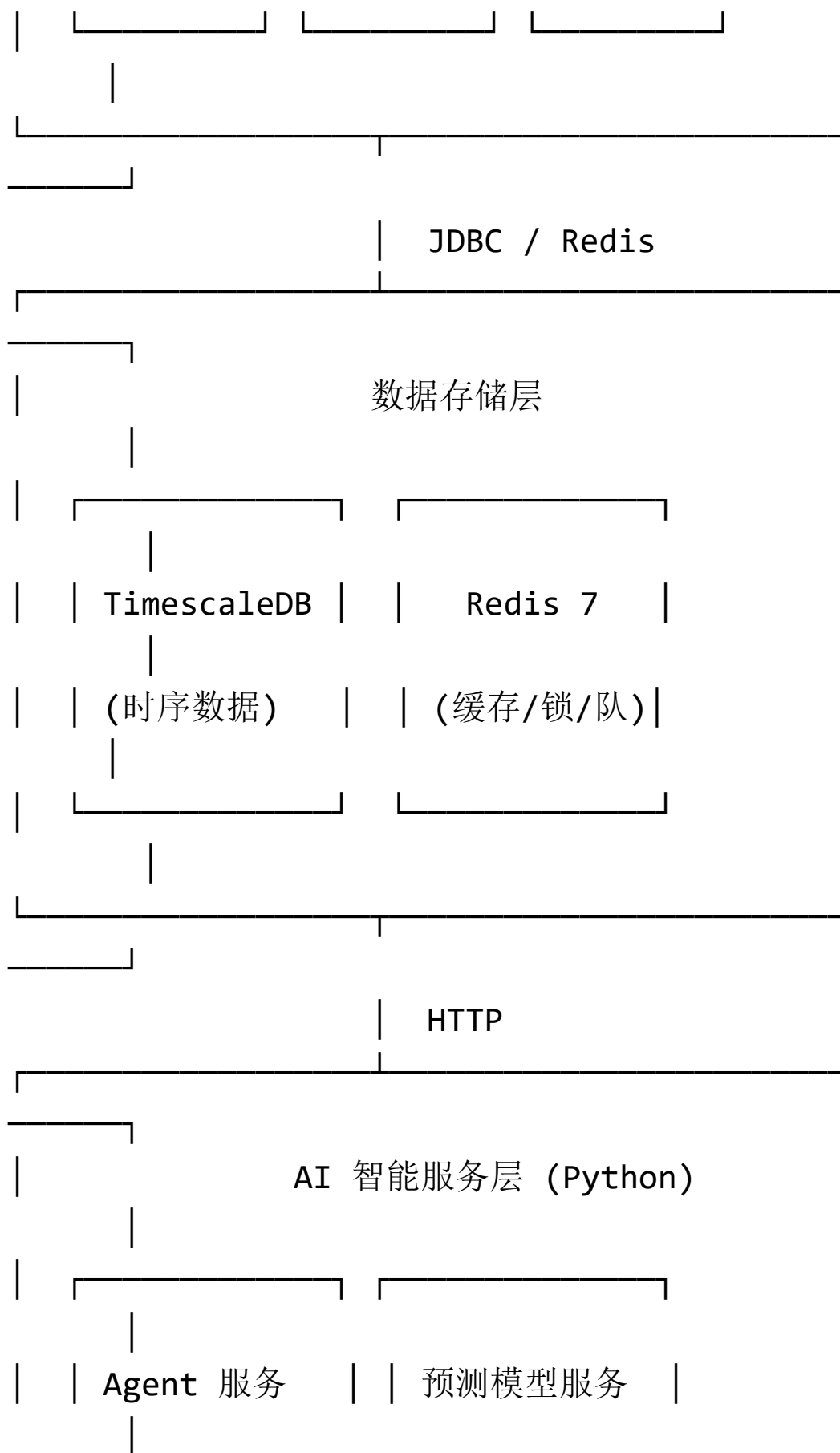
3 AI 辅助开发工具

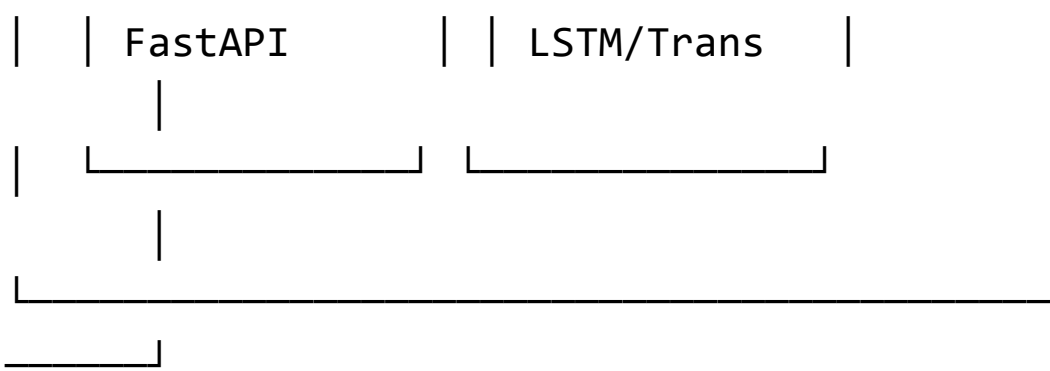
工具	用途	注意事项
GitHub Copilot	代码补全、重构建议	需验证生成代码的准确性和安全性
Claude Code	AI辅助编码、代码审查	用于生成模板代码、编写文档、接口调试
DeepSeek API	LLM故障诊断、Agent推理	作为系统运行的AI引擎，非开发辅助工具

4 技术架构图









5 技术风险与应对措施

风险	等级	应对措施
LLM API 调用延迟 >30s	中	设置超时降级、异步调用、预设模板兜底
TimescaleDB 超表查询性能下降	低	合理建索引、使用连续聚合、数据压缩策略
Redis 内存溢出	低	设置 maxmemory + 淘汰策略（allkeys-lru）
Python AI 服务与 Java 后端通信失败	中	健康检查 + 重试机制 + 降级策略
Vue 3 响应式性能问题	低	虚拟滚动、懒加载、ECharts 按需引入

6 附则

- 本文档由开发团队共同维护，技术栈调整需经全队评审；
- 文档修订需同步归档，确保技术选型的时效性与准确性；
- 所有第三方依赖需关注许可证合规性（优先选用 MIT/Apache 2.0 许可）。